

Decision Trees

DSC 340 Applied Machine Learning

EMC 7.1, PDSH 5.08

March 10

The Plan

- Decision trees
- Revisit supervised learning process
- Examples: medical diagnosis and IRIS
- Assessment and variations

Looking Ahead

- Wed 3/12 Test 3
- Fri 3/14 Random Decision Forests
- Mon 3/17 Support Vector Machines
Survey: new partner assignments
- Wed 3/19* Start Lab 4 (with new partner)
- Fri 3/21* Behold, the Perceptron
- Then ... Spring Break

Review: Supervised Learning

Let's make sure this is clear!

Review: Supervised Learning

- Given **training set** of N examples:
 - $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)$
- where each y_i generated by **unknown function** $y = f(\mathbf{x})$
- discover **hypothesis (or model)** h that **approximates true function** f .
- In other words, h should correctly predict new examples (**test set**).
- **Hypothesis space** $\mathcal{H}$
- Hypothesis $h \in \mathcal{H}$ is **consistent** if it agrees with the data
- **Occam's razor**

Review: ML Process

1. Define the problem
2. Gather the data
3. Prepare the data
4. Choose a model
5. Train the model
6. Evaluate the model
7. Tune hyperparameters
8. Make predictions

Missing data, encoding, scaling

Set of models, hyperparameters

Train-test split; fit each model

On a validation set

Go to 4; select best model overall

Evaluate best model on test set

Decision Trees

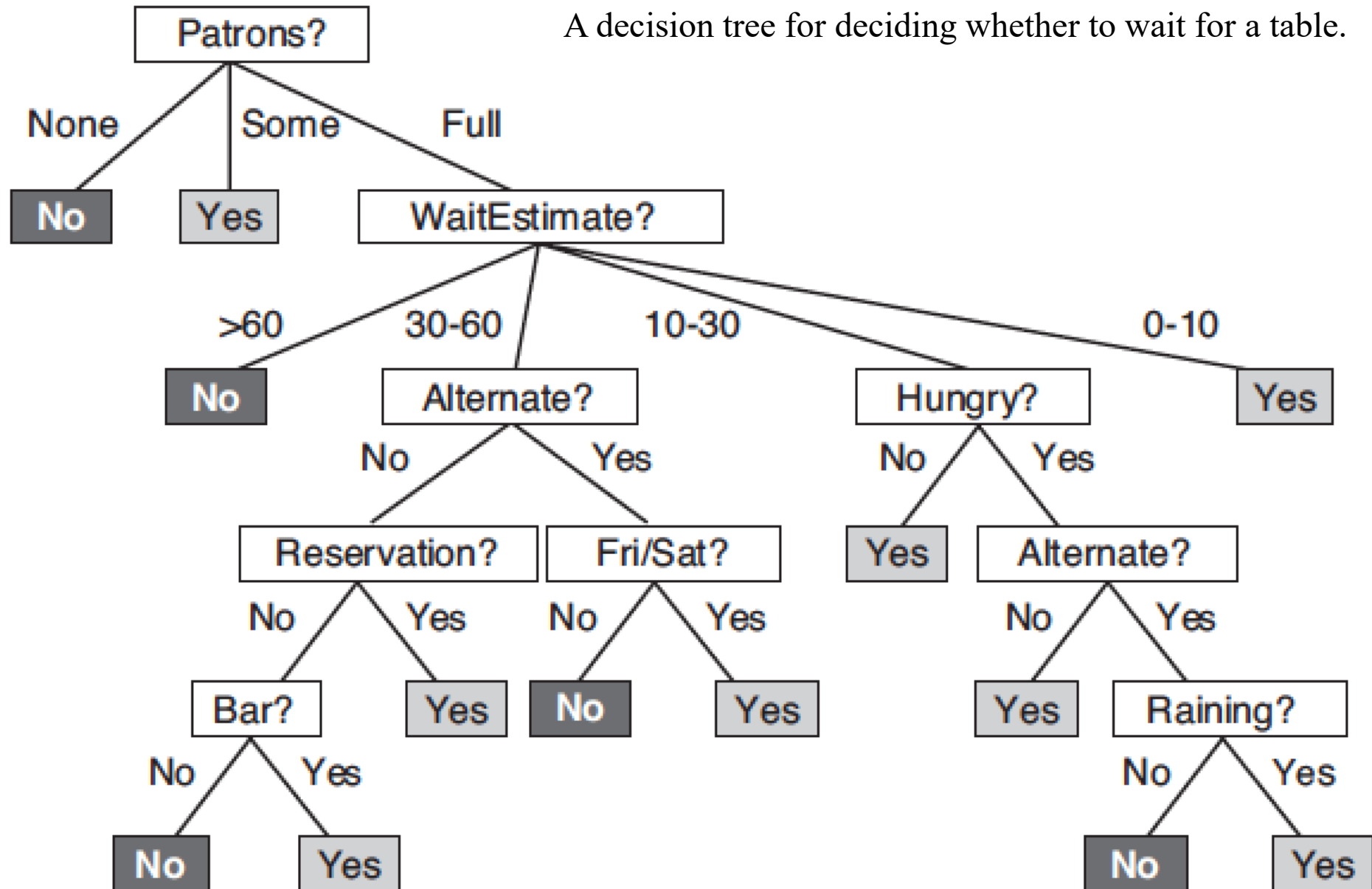
(We are working our way toward random forests)

Decision Tree

- Simple function
- **Input:** vector of attribute values $x = \langle x_1, x_2, \dots, x_n \rangle$
- **Output:** target y
- Values (both x and y) can be **numeric** (int, real) or **categorical**
- **Implemented with nothing but nested if-else statements**
- Basically a flow chart
- But no loops, no recursion, no funny stuff

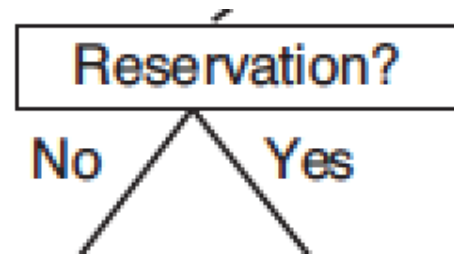
Example: R&N AI text book

A decision tree for deciding whether to wait for a table.

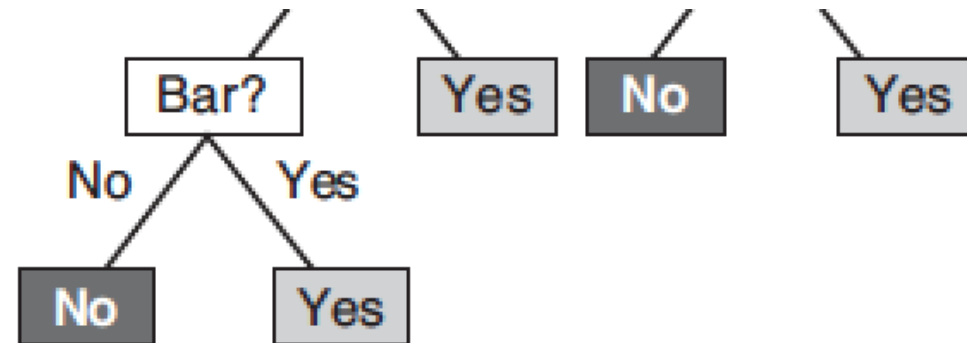


Decision Tree

- Interior nodes **test value** of some **attribute**

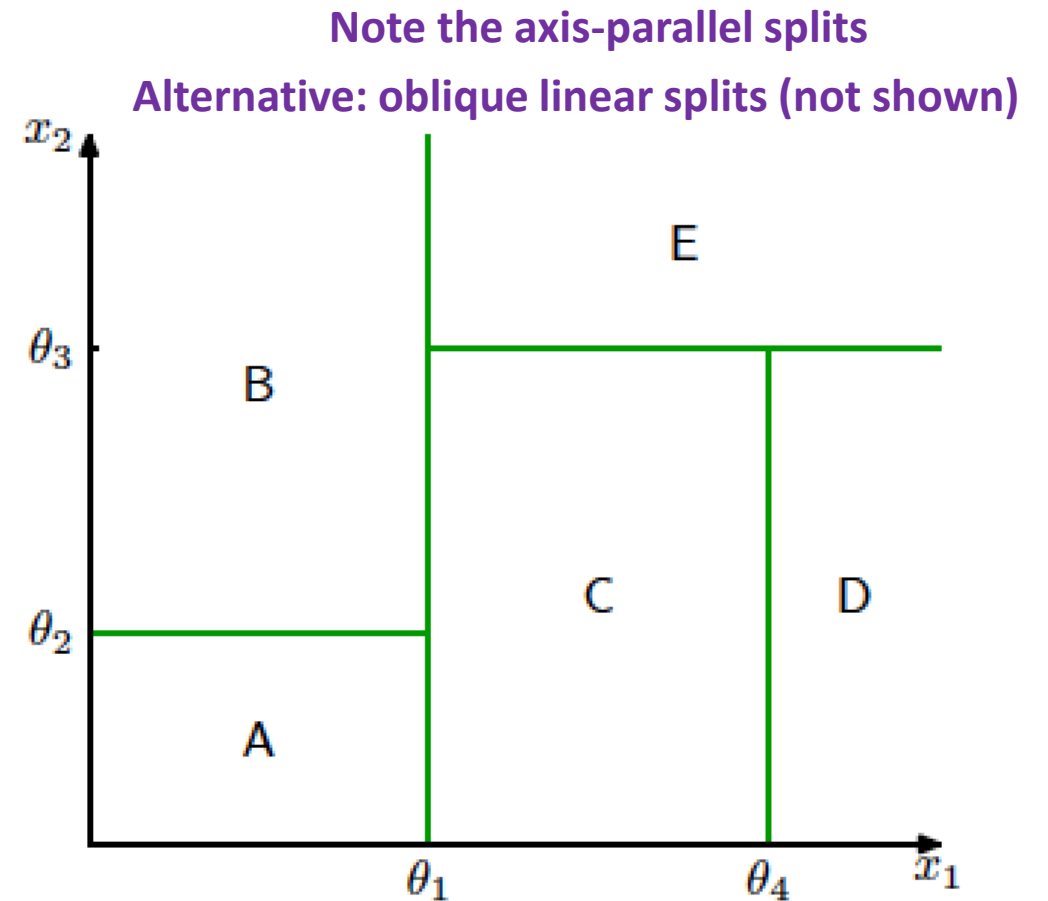
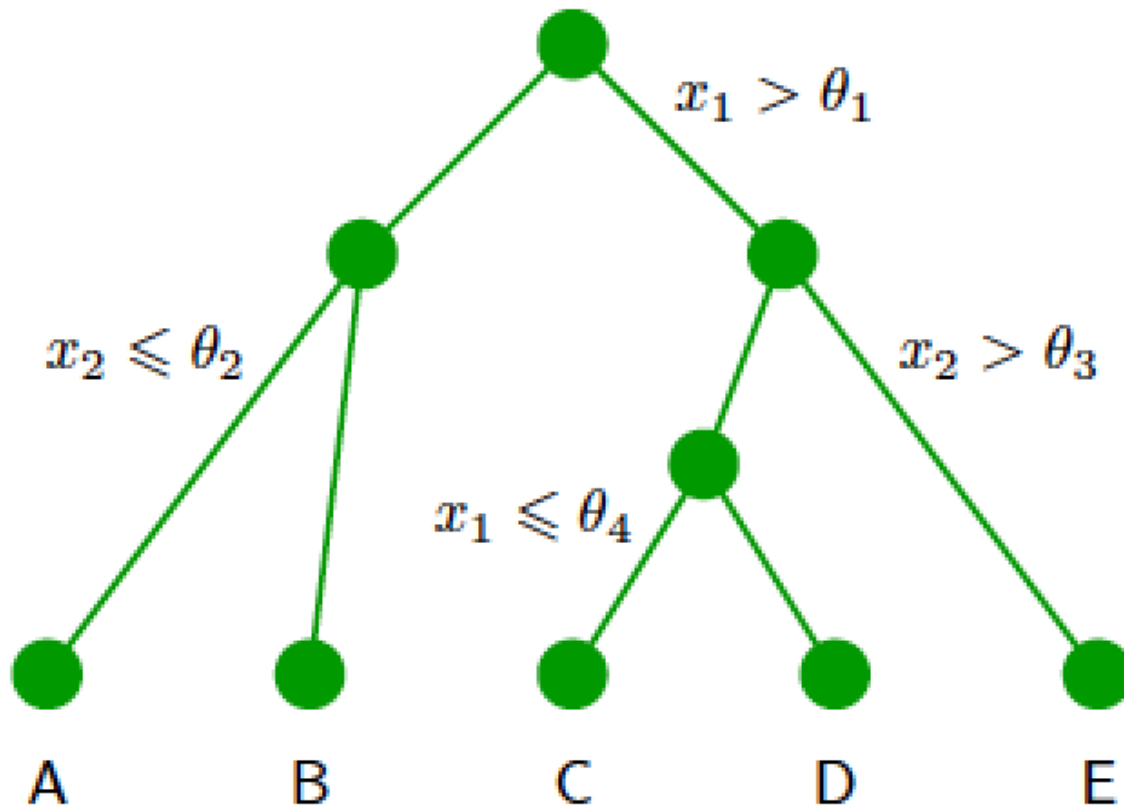


- Leaves represent **decisions**



- As we descend, **successively partition space** of possible inputs x
- Eventually settle on a decision y

Mathematical basis



Bishop, Pattern Recognition and Machine Learning (Fig. 14.5)

Learning Decision Trees

Learning Decision Trees

- **Example data** consisting of **input-output pairs**
 - **Input** is a matrix X
 - Each **row** is a vector of attribute values
- Assume classification: Known **class labels** y
- Find a decision tree T that **minimizes height**
- **NP-hard**: we'd have to search through 2^{2^n} trees!
- Use **greedy** recursive algorithm (divide-and-conquer)
- Each time: Divide on "**most important**" **attribute**; recurse

Metrics for Choosing Each "Split"

Choose the "split" that:

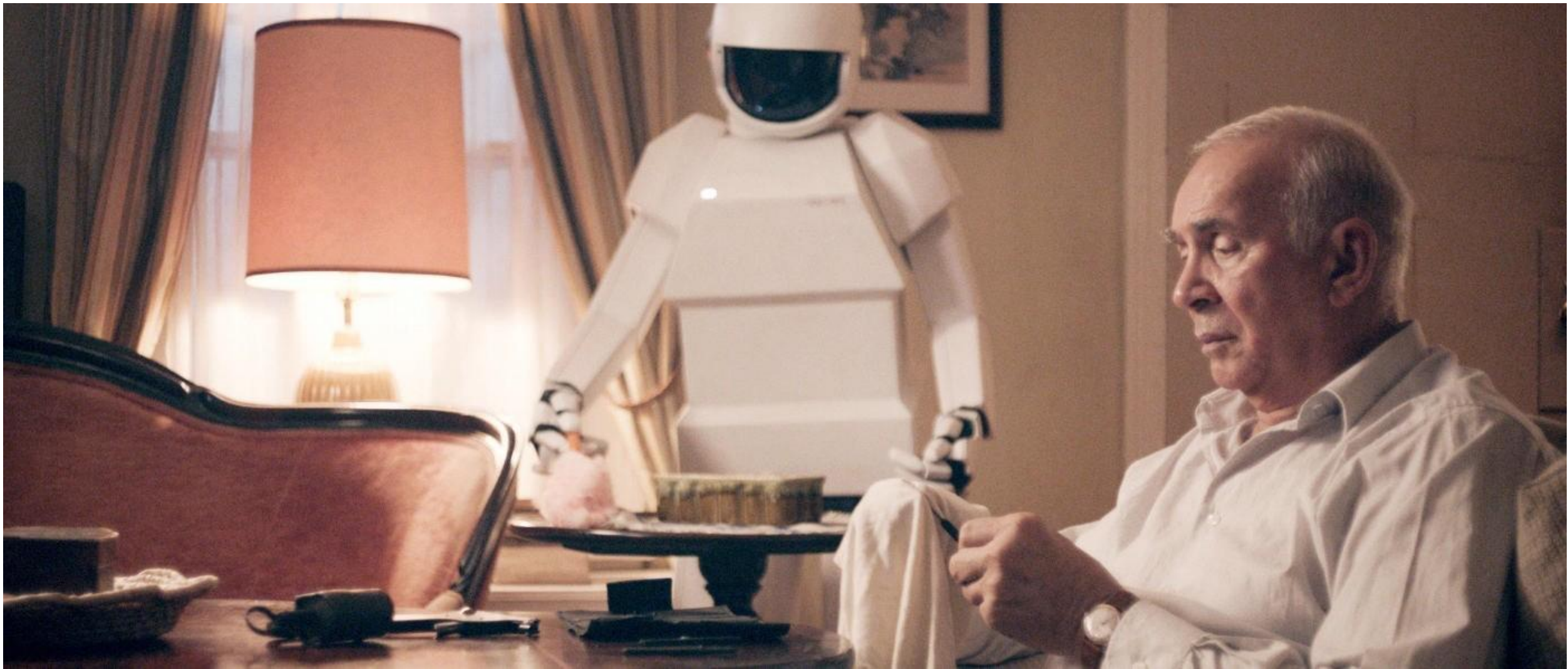
- Maximizes **information gain**
- Minimizes **entropy**
 - **Gini impurity:** common proxy for entropy
 - "Measure of how often a randomly chosen element from the set would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the subset."
 - It reaches its minimum (zero) when all cases in the node fall into a single target category.
 - Not to be confused with Gini coefficient in economics (inspired by, not same)

Example: Medical Diagnosis

Scikit-Learn Tree Module and Acute Inflammations Dataset

ML Process

- 1. Define the problem**
2. Gather the data
3. Prepare the data
4. Choose a model
5. Train the model
6. Evaluate the model
7. Tune hyperparameters
8. Make predictions



<http://lisamyerslisamyers.com/2014/01/11/robot-and-frank/>

1. Define the Problem

- Context: smart home to support elderly
- Resident is experiencing some discomfort
- Should the house seek medical attention?
- Specific example: cystitis and nephritis
 - Cystitis = inflammation of urinary bladder (usu. painful!)
 - Nephritis = inflammation of kidneys
 - Related problems, overlapping symptoms, may occur together

ML Process

1. Define the problem
- 2. Gather the data**
3. Prepare the data
4. Choose a model
5. Train the model
6. Evaluate the model
7. Tune hyperparameters
8. Make predictions

2. Gather the Data

- Collect data from medical records
- Thankfully, for our purposes, we have a cultivated dataset:
 - <http://archive.ics.uci.edu/ml/datasets/acute+inflammations>
- Files: diagnosis.data

35, 5	no	yes	no	no	no	no	no
35, 9	no	no	yes	yes	yes	yes	no
35, 9	no	yes	no	no	no	no	no
36, 0	no	no	yes	yes	yes	yes	no
36, 0	no	yes	no	no	no	no	no
36, 0	no	yes	no	no	no	no	no

Attribute Information:

- a1 Temperature of patient { 35C-42C }
- a2 Occurrence of nausea { yes, no }
- a3 Lumbar pain { yes, no }
- a4 Urine pushing (continuous need for urination) { yes, no }
- a5 Micturition pains { yes, no }
- a6 Burning of urethra, itch, swelling of urethra outlet { yes, no }
- d1 decision: Inflammation of urinary bladder { yes, no }
- d2 decision: Nephritis of renal pelvis origin { yes, no }

ML Process

1. Define the problem
2. Gather the data
- 3. Prepare the data**
4. Choose a model
5. Train the model
6. Evaluate the model
7. Tune hyperparameters
8. Make predictions

3. Prepare the Data

- Read the file: remove extraneous characters, formatting issues ☹️
- Data wrangling or data munging
- Read into NumPy arrays X and y
- Randomly shuffle, partition into training and testing sets (80/20)

```
X_data, y_data = load_diagnosis()
```

```
feature_names = ['Temperature',
                  'Nausea',
                  'Lumbar pain',
                  'Urine pushing',
                  'Micturition',
                  'Urethral discomfort']
```

```
class_names = ['¬cystitis, ¬nephritis',
               '¬cystitis, nephritis',
               'cystitis, ¬nephritis',
               'cystitis, nephritis']
```

[illegible]

```
[ [37.5  0.   0.   1.   1.   1. ]  
  [36.6  0.   1.   0.   0.   0. ]  
  [36.7  0.   1.   0.   0.   0. ]  
  [41.3  1.   1.   1.   1.   0. ]  
  [40.   1.   1.   0.   1.   0. ]  
  [40.8  0.   1.   1.   0.   1. ] ... ]
```

```
[2 0 0 3 1 1 ...]
```

ML Process

1. Define the problem
2. Gather the data
3. Prepare the data
- 4. Choose a model**
5. Train the model
6. Evaluate the model
7. Tune hyperparameters
8. Make predictions

ML Process

1. Define the problem
2. Gather the data
3. Prepare the data
4. Choose a model
- 5. Train the model**
6. Evaluate the model
7. Tune hyperparameters
8. Make predictions

```
# LEARN MODEL
```

```
clf = tree.DecisionTreeClassifier()
```

```
clf = clf.fit(X_train, y_train)
```

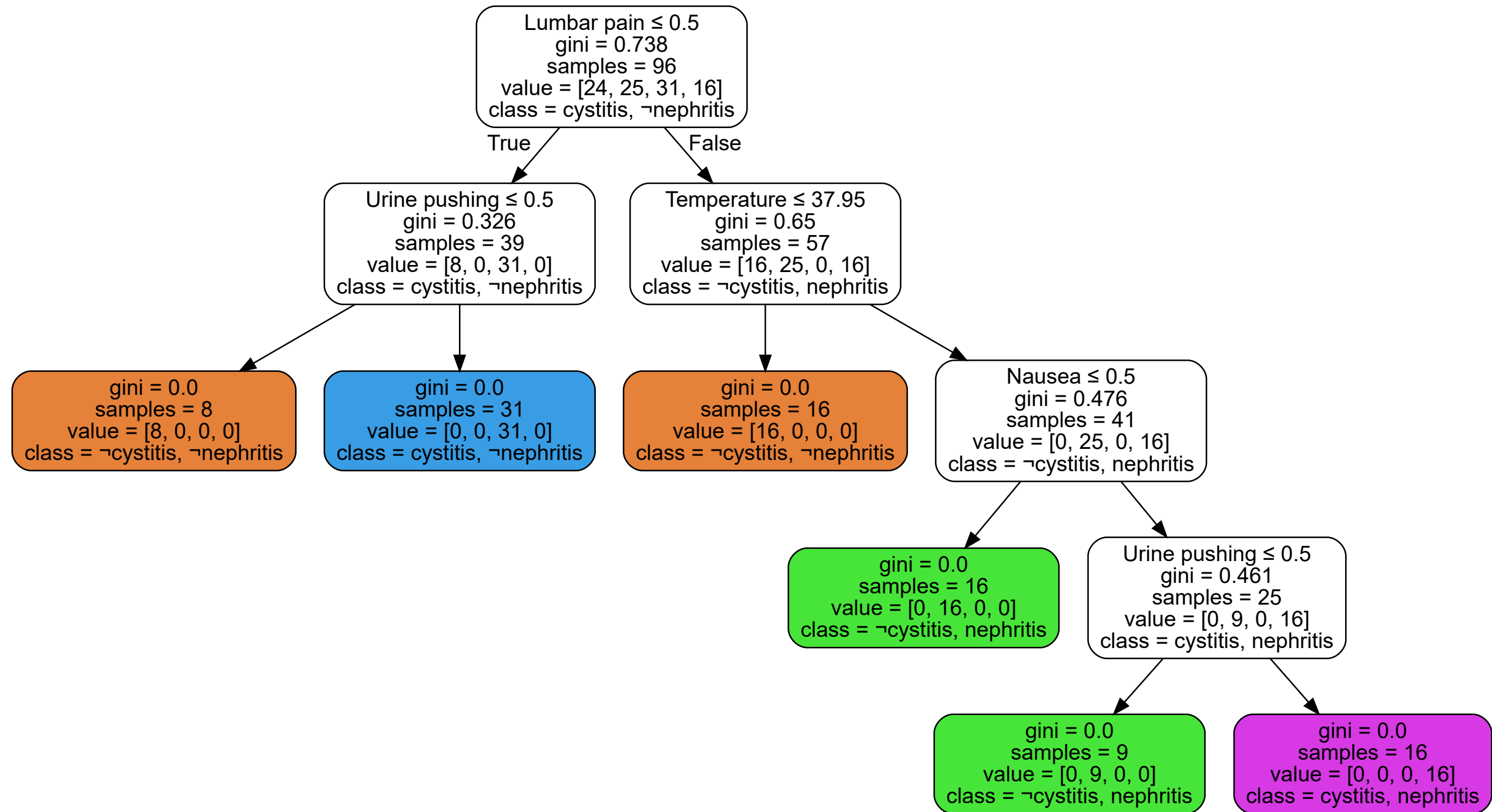
```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,  
                        max_features=None, max_leaf_nodes=None,  
                        min_impurity_decrease=0.0, min_impurity_split=None,  
                        min_samples_leaf=1, min_samples_split=2,  
                        min_weight_fraction_leaf=0.0, presort=False, random_state=42,  
                        splitter='best')
```

```
# VISUALIZE RESULTS
```

```
dot_data = tree.export_graphviz(clf, out_file=None,  
                                feature_names=feature_names,  
                                class_names=class_names,  
                                filled=True, rounded=True,  
                                special_characters=True)
```

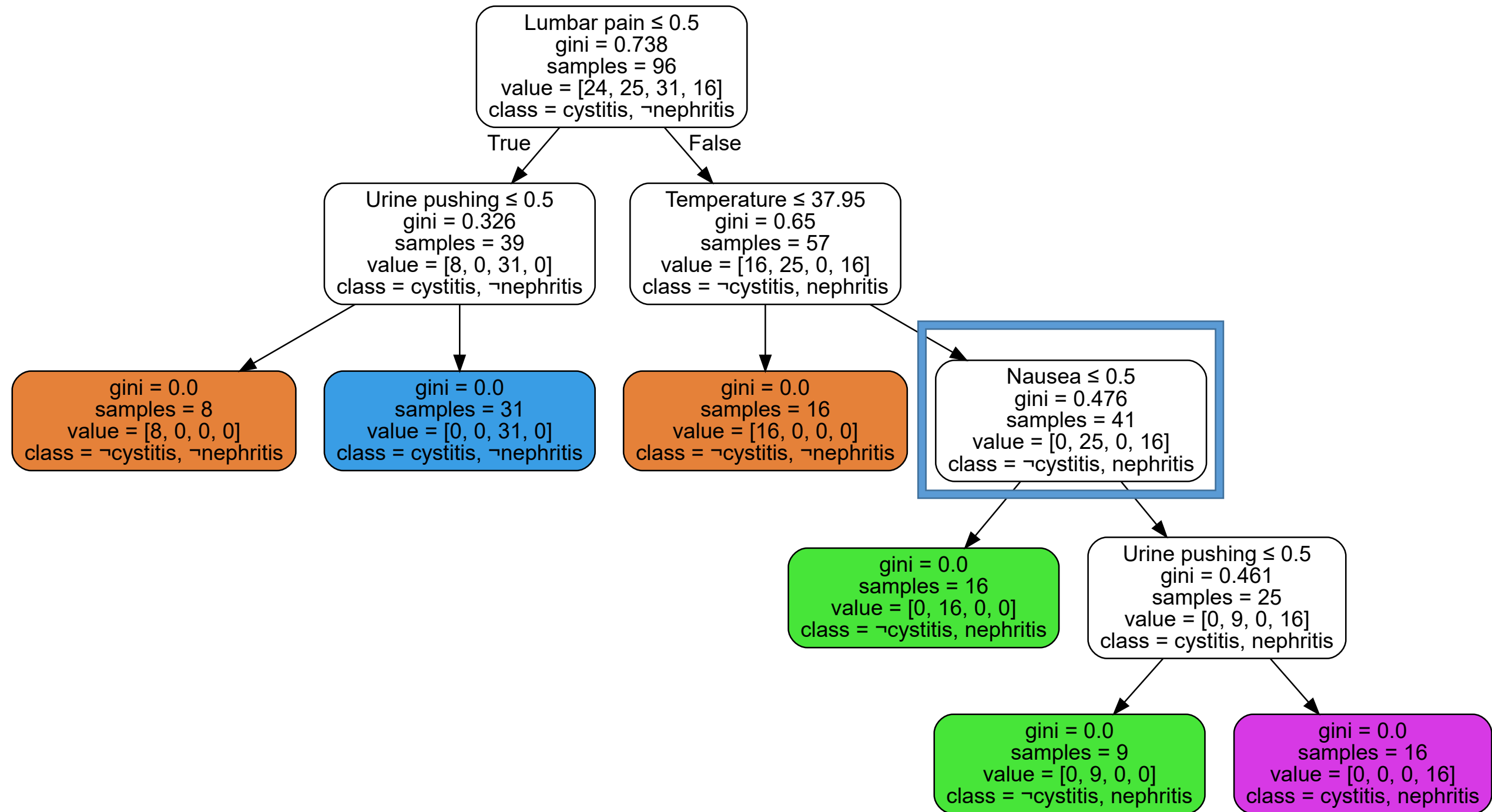
```
graph = graphviz.Source(dot_data)
```

```
graph.render('diagnosis') # Write to diagnosis.pdf
```



Gini Impurity

- Calculation to determine best split (attribute/value)
- Score between 0 and 1 (1=best, 0=worst).
- Algorithm chooses indicator with highest value to split node
- Moves on to the next node and repeats ...



Computing Gini Impurity

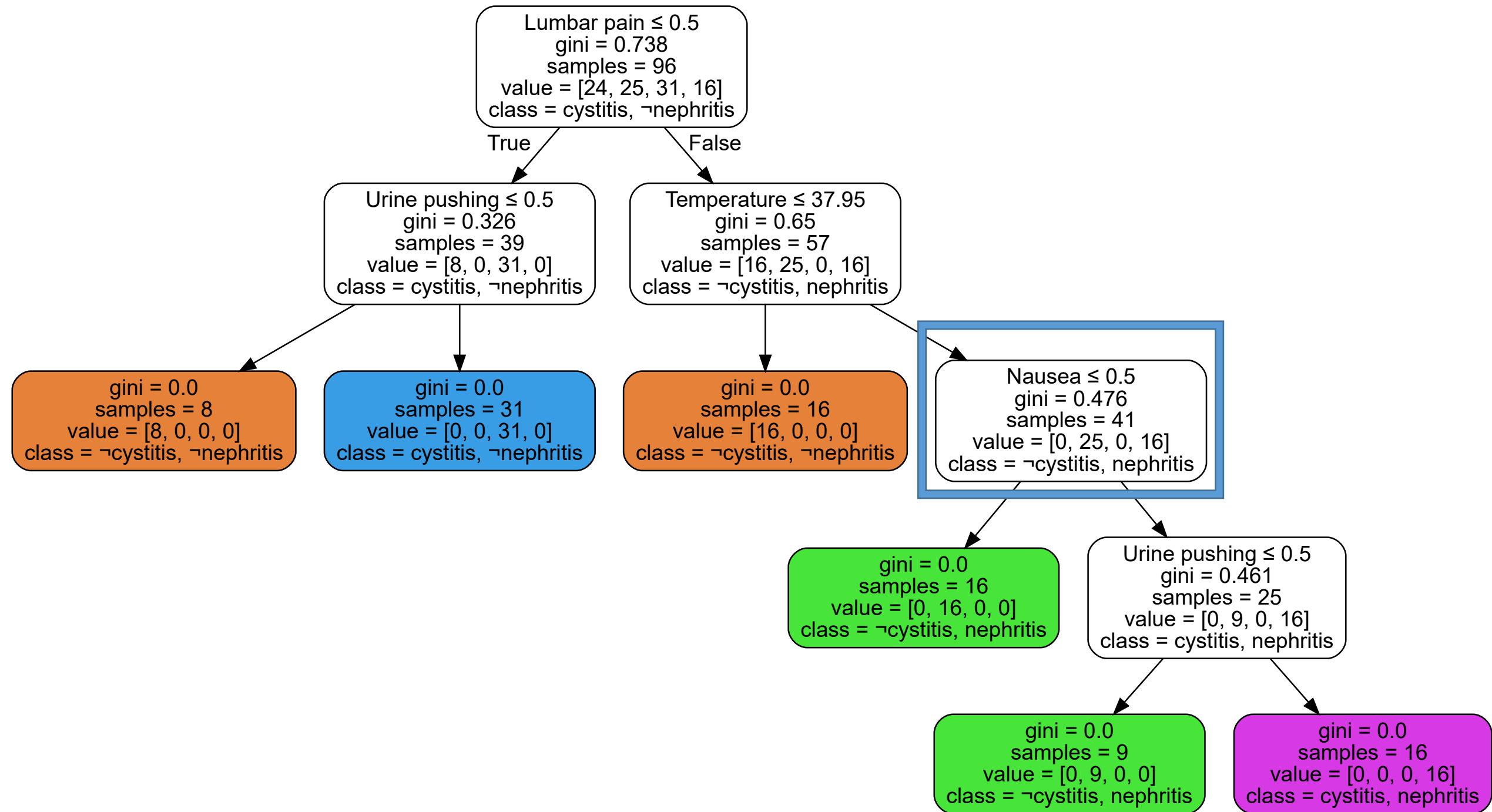
- Computation: $I_G(p) = 1 - \sum_{i=1}^J p^2$
 - J = number of classes
 - p_i = fraction of items labeled class i ,
where $i \in \{1, \dots, J\}$

```
# buckets and counts as shown
values = [0, 25, 0, 16]

# min-max scale values
p = [count / sum(values)
      for count in values]

# compute gini impurity score
impurity = 1
for p_i in p:
    impurity -= p_i ** 2

print(impurity) # 0.476 :)
```



Computing Gini Impurity

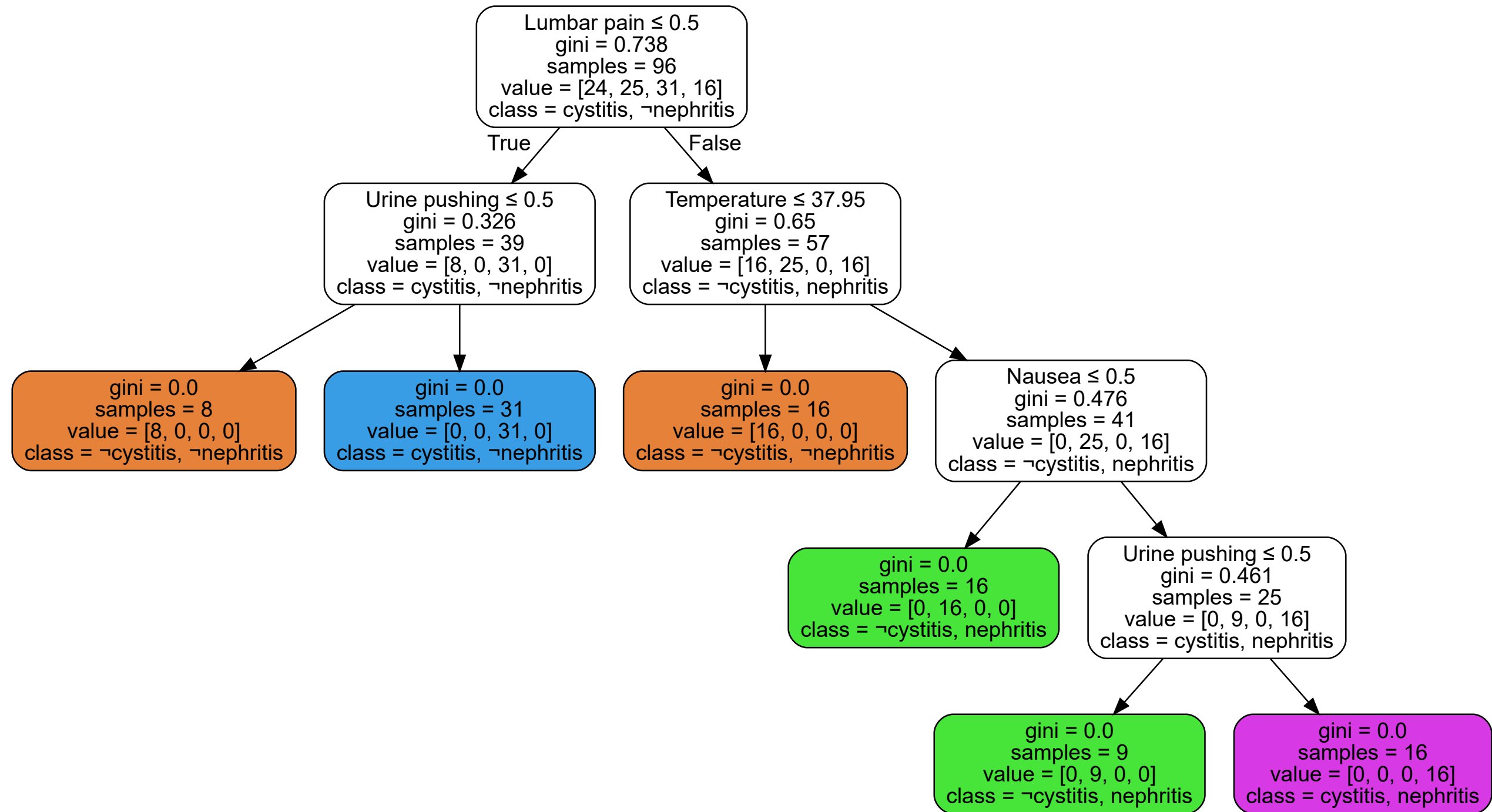
- Computation: $I_G(p) = 1 - \sum_{i=1}^J p^2$
 - J = number of classes
 - p_i = fraction of items labeled class i ,
where $i \in \{1, \dots, J\}$
- Not to be confused with **Gini coefficient**
- Alternative measures include **entropy**
information gain:
 - $H(T) = I_E(p_1, p_2, \dots, p_J) = -\sum_{i=1}^J p_i \log_2 p_i$

```
# buckets and counts as shown
values = [0, 25, 0, 16]
```

```
# min-max scale values
p = [count / sum(values)
      for count in values]
```

```
# compute Gini impurity score
impurity = 1
for p_i in p:
    impurity -= p_i ** 2

print(impurity) # 0.476 :)
```



ML Process

1. Define the problem
2. Gather the data
3. Prepare the data
4. Choose a model
5. Train the model
- 6. Evaluate the model**
7. Tune hyperparameters
8. Make predictions

6. Evaluate the Model

- Recall: **hypothesis h approximates true function f**
- That is, h should correctly predict new examples ...
- Our decision tree is a hypothesis.
- We now apply it to our **test set**.
- Count how often predicted value $h(x) = y$
- Predicted value uses the decision tree
- Compare that to "ground truth" (known class label) y

```
# EVALUATE MODEL
```

```
y_predict = clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_predict) * 100
```

```
print("Accuracy is %.1f percent" % accuracy)
```

Predictions:

37.5 0 0 1 0 0 2 2

37.6 0 0 1 1 1 2 2

36.0 0 1 0 0 0 0 0

37.9 0 0 1 1 0 2 2

.

Accuracy is 100.0 percent

4. Evaluate the Model

- Note: You should always be suspicious of 100% accuracy!
- For this problem, usu. 100%, but not always (unless seeded at 42!):

41.0	1	1	1	1	1	3	3
------	---	---	---	---	---	---	---

37.9	0	1	0	0	0	0	1
------	---	---	---	---	---	---	---

40.2	0	0	0	0	0	0	0
------	---	---	---	---	---	---	---

41.0	0	0	0	0	0	0	0
------	---	---	---	---	---	---	---

37.0	0	0	1	0	0	2	2
------	---	---	---	---	---	---	---

37.7	0	0	1	1	0	2	2
------	---	---	---	---	---	---	---

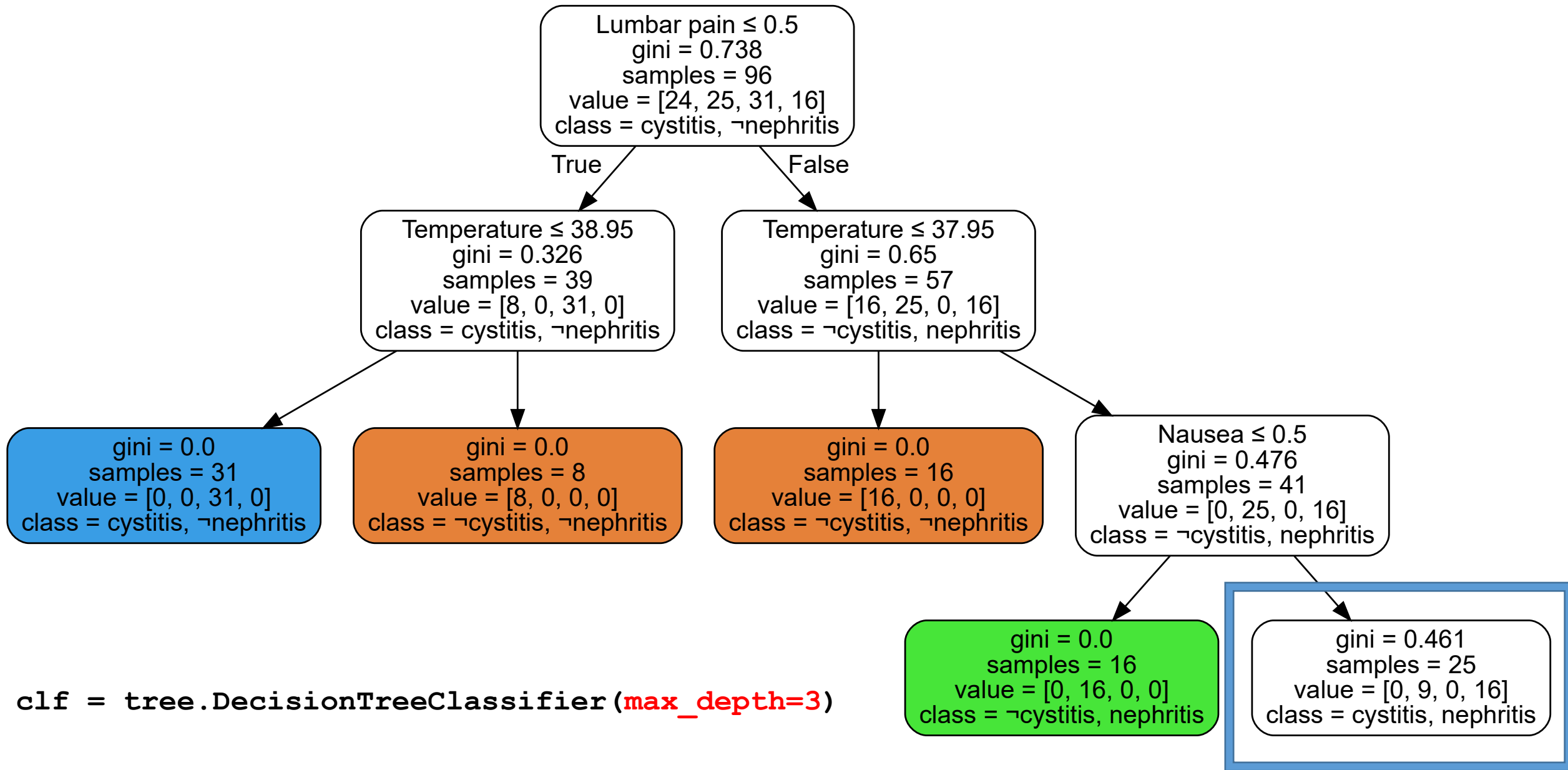
Accuracy is 95.8 percent

ML Process

1. Define the problem
2. Gather the data
3. Prepare the data
4. Choose a model
5. Train the model
6. Evaluate the model
- 7. Tune hyperparameters**
8. Make predictions

7. Tune Hyperparameters

- `max_depth` how deep tree can be— i.e., # of splits
- `min_samples_split` min. # samples to split internal node
- `min_samples_leaf` min. # samples to be at a leaf node
- `max_features` # features to consider when finding best split
- `criterion` gini or entropy (max information gain)
- `class_weight` can assign weights to the classes
- There are others ...



```
clf = tree.DecisionTreeClassifier(max_depth=3)
```

Accuracy is 95.8 percent

ML Process

1. Define the problem
2. Gather the data
3. Prepare the data
4. Choose a model
5. Train the model
6. Evaluate the model
7. Tune hyperparameters
8. **Make predictions**



<http://lisamyerslisamyers.com/2014/01/11/robot-and-frank/>

Example: Iris Data Set

Another example

CLASSIFY IRISES



Setosa

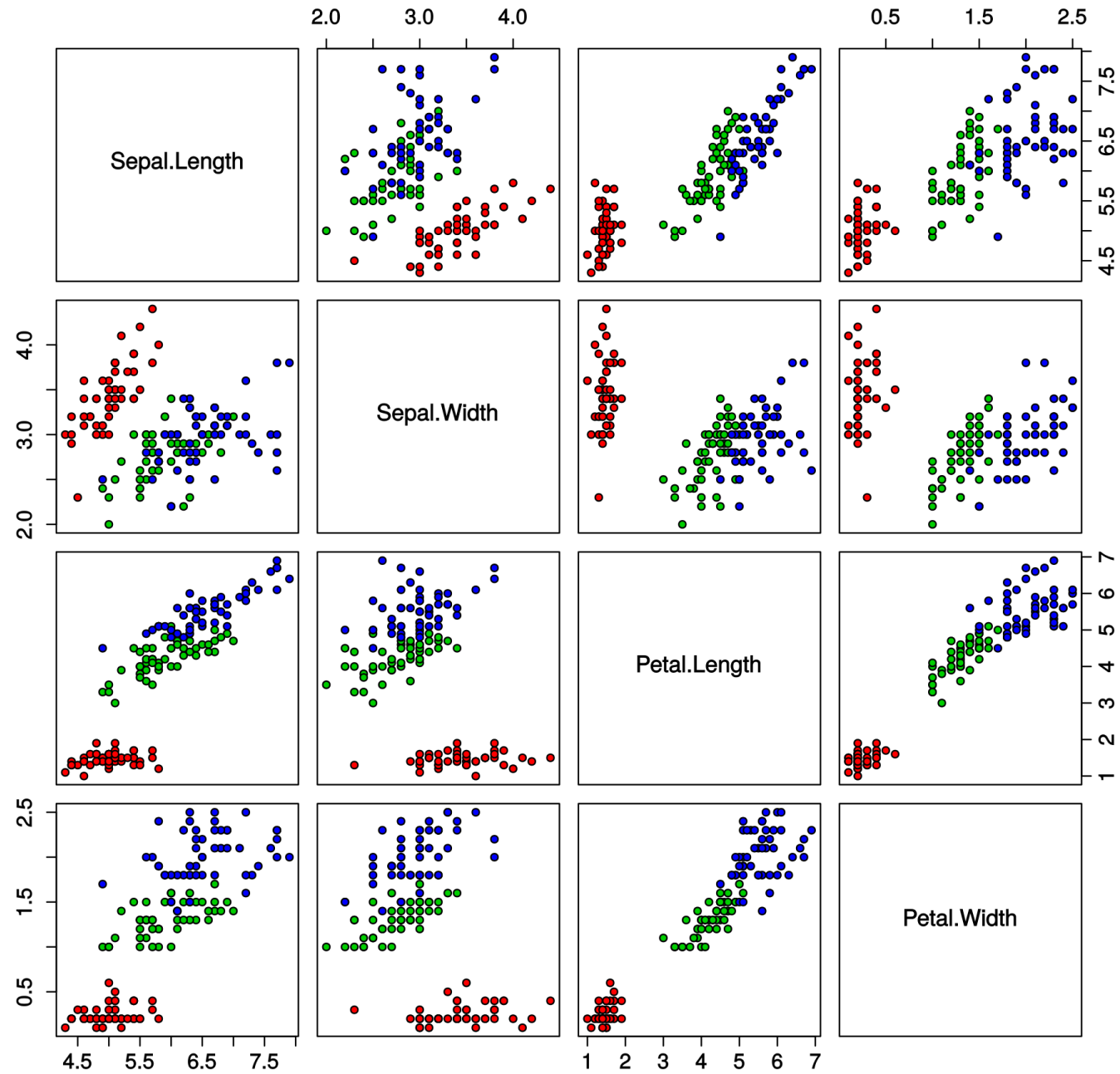


Versicolor



Virginica

Iris Data (red=setosa,green=versicolor,blue=virginica)



```
# PREPARE DATA
```

```
iris = load_iris() # built-in dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(iris.data,  
                                                    iris.target,  
                                                    test_size=0.20,  
                                                    random_state=0)
```

```
# CLASSIFY DATA
```

```
clf = tree.DecisionTreeClassifier() # use decision tree model
```

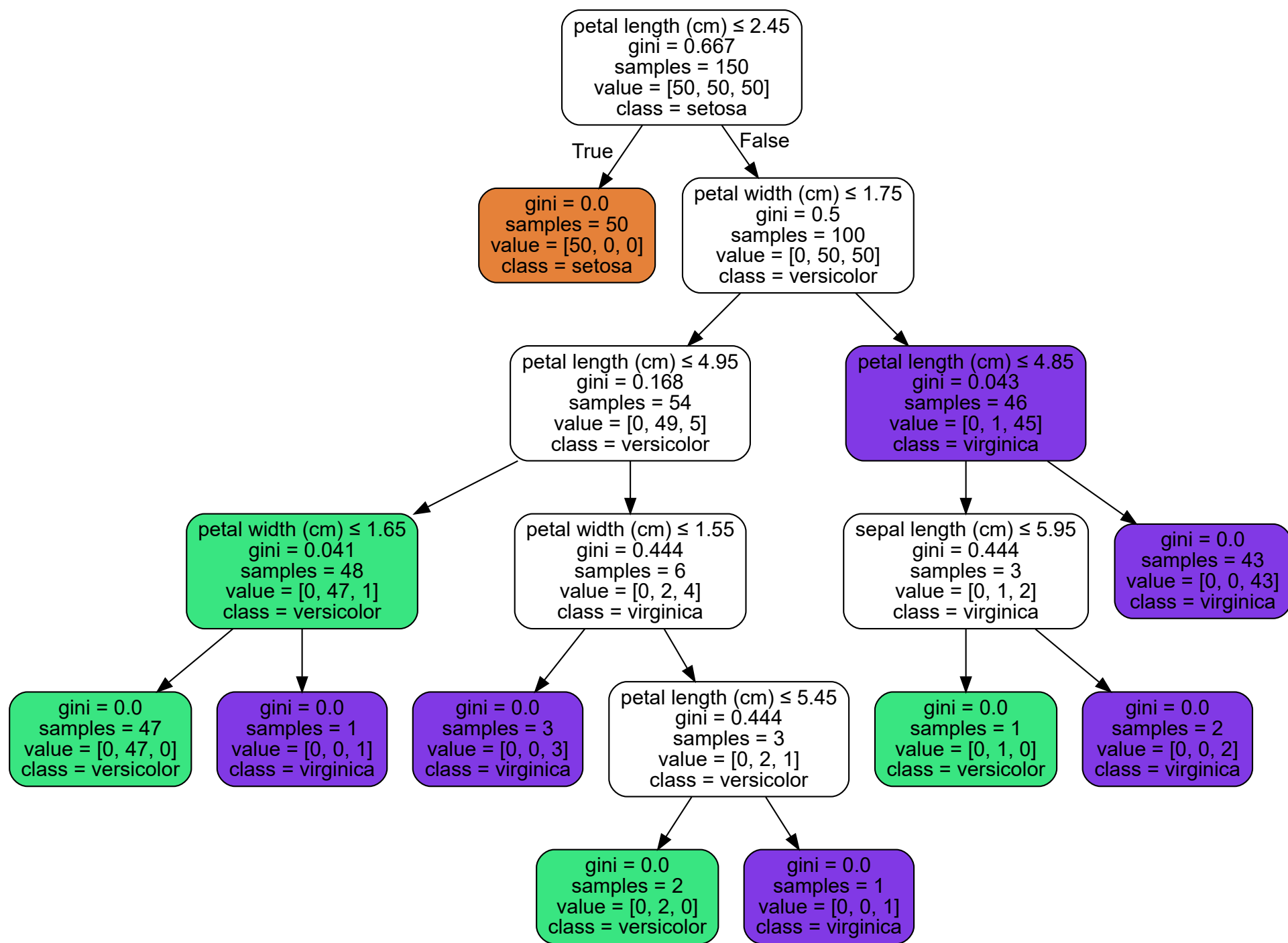
```
clf = clf.fit(X_train, y_train) # fit model to data
```

```
# VISUALIZE RESULTS
```

```
dot_data = tree.export_graphviz(clf, out_file=None,  
                                feature_names=iris.feature_names,  
                                class_names=iris.target_names,  
                                filled=True, rounded=True,  
                                special_characters=True)
```

```
graph = graphviz.Source(dot_data)
```

```
graph.render('iris') # now open iris.pdf manually
```



Discussion

There are (many!) other models ...

Decision Trees

The Good

- Easy to understand and interpret
- Useful even if limited hard data
- Shows worst, best and expected values for different scenarios
- Transparent (glass box) model
- Can combine with other decision techniques

The Not-So-Good

- Unstable (small change in data can lead to big change in tree)
- Less accurate than other models
- Can get big if we don't set limits
- Lots of hyperparameter tuning
- Can be complex to use if lot of uncertainty or many outcomes

Variations

- **Regression trees** Take mean of values in leaves
- **CART** Classification and Regression Tree
- **Model trees** Linear regression model, etc., at each leaf (!)
- **Random forest** next time ...